TEYZIX CORE

Internship Portal

Project Type:	**Full Stack MERN Web Application**
Student Name:	**Zeeshan Ahmad**
Institution:	**UET Lahore – Computer Engineering**
Submission Date:	**May 15, 2026**

MongoDB | Express.js | React | Node.js | Tailwind CSS | Resend API

Bridging talent and opportunity through modern technology

. TABLE OF CONTENTS

1. INTRODUCTION

■ What is an Internship Portal?

An Internship Portal is a centralized digital platform where organizations publish available positions and students submit their applications. It functions as a professional bridge between emerging talent and career opportunity, eliminating the inefficiencies of scattered, manual processes.

■ Why is This System Needed?

Traditional application tracking via spreadsheets or email chains is inherently error-prone, difficult to scale, and lacks any meaningful user experience for applicants. A dedicated portal guarantees **data integrity, structured communication, and real-time administrative oversight**.

■ Problem Statement

Conventional internship application methods involve fragmented communication channels and offer no real-time visibility for administrators. This leads to lost applications, delayed responses, and an unprofessional impression on prospective interns.

■ Objectives

- To provide a professional glassmorphic UI for applicants with a smooth application experience.
- To implement a secure, validated backend for reliable data storage on MongoDB Atlas.
- To automate administrator notifications for every new submission via the Resend API.
- To create a filterable, searchable admin dashboard for efficient application management.
- To deploy a production-ready full-stack application accessible globally.

2. EXISTING VS. PROPOSED SYSTEM

Feature	Existing System (Manual)	Proposed System (TEYZIX)
Data Storage	Spreadsheets / Paper	**MongoDB Atlas (Cloud)**
Notifications	Manual Email	**Automated via Resend API**
Accessibility	Local / Limited	**Global (Netlify + Railway)**
Security	Low – Public Files	**Password-Protected Admin**
UI / UX	Non-existent	**Modern Glassmorphism**
Scalability	Poor	**Horizontally Scalable**
Data Integrity	Prone to errors	**Schema Validation (Mongoose)**

3. PROJECT SCOPE

The scope of the TEYZIX CORE Internship Portal encompasses the full design, development, testing, and deployment of a production-grade MERN application. The deliverables include:

- Public-facing internship listings with detailed domain descriptions.
- A multi-field application form with real-time client-side and server-side validation.
- MongoDB Atlas cloud integration for persistent, reliable data storage.
- Automated email alert system using the Resend API upon each new application.
- A secure administrative interface protected by password authentication.
- Search, filter, and detail-view capabilities for the admin dashboard.
- Full responsive design targeting mobile, tablet, and desktop viewports.
- Deployment pipeline with Netlify (frontend) and Railway (backend).

4. TECHNOLOGIES USED

Technology	Version / Tool	Purpose
React	v18.x + Vite	Frontend SPA Framework
Node.js	v20.x LTS	Backend Runtime Environment
Express.js	v4.x	RESTful API Server
MongoDB Atlas	Cloud (M0 Tier)	NoSQL Database
Mongoose	v8.x	ODM / Schema Validation
Tailwind CSS	v4.x	Utility-First CSS Framework
Axios	v1.x	HTTP Client (Frontend)
Resend API	REST API	Automated Email Notifications
Netlify	CDN Platform	Frontend Deployment
Railway	Cloud Platform	Backend Deployment
Postman	v11.x	API Testing & Documentation

5. SYSTEM ARCHITECTURE

■ Full-Stack Workflow Overview

The application follows a classic **client-server architecture** with clear separation of concerns. The React frontend communicates exclusively via HTTP with the Express backend, which handles validation, database operations, and third-party API calls.

1	User fills the application form on the React frontend.
2	Axios sends an HTTP POST request to the Express server.
3	Express validates the payload and saves data to MongoDB Atlas.
4	Resend API is triggered to dispatch an email alert to the administrator.
5	The server responds with a 201 Created status to confirm submission.
6	Admin logs in to the protected dashboard via password authentication.
7	Dashboard fetches all applications via a GET request and renders them.

■ API Request Sequence

Every form submission follows a synchronous request-response cycle: **Frontend** → **Validation** → **Database** → **Email** → **Response**. The Admin dashboard operates over a separate authenticated GET flow.

6. DATABASE DESIGN (MongoDB)

■ Collection: applications

The primary data model is stored in the **applications** collection within MongoDB Atlas. Mongoose enforces the schema at the application layer, ensuring data consistency.

Field	Type	Constraints	Description
`_id`	ObjectId	Auto-generated	Unique record identifier
`name`	String	Required, trimmed	Full name of the applicant
`email`	String	Required, validated	Contact email address
`phone`	String	Required	Applicant phone number
`domain`	String	Required, enum	Internship category/track
`message`	String	Required, max 500	Cover letter / message
`createdAt`	Date	Auto (timestamps)	Submission timestamp

7. API DOCUMENTATION

Submit Application	
Method	**POST**
Endpoint	`/api/applications`
Access	Public
Request Body	`{ name, email, phone, domain, message }`
Success	201 Created – Application saved & email dispatched
Errors	400 Bad Request – Validation failure | 500 Server Error

Get All Applications	
Method	**GET**
Endpoint	`/api/applications/all`
Access	Admin Only
Request Body	None
Success	200 OK – Array of application objects
Errors	401 Unauthorized | 500 Server Error

Admin Login	
Method	**POST**
Endpoint	`/api/applications/login`
Access	Public
Request Body	`{ password }`
Success	200 OK – Access granted
Errors	401 Unauthorized – Incorrect password

8. FRONTEND DOCUMENTATION

■ Pages & Components

Page / Component	Route	Description
Home Page	`/`	Landing page with hero section, intro, CTA
Internships Page	`/internships`	Lists available internship domains with cards
Apply Page	`/apply`	Multi-field application form with validation
Admin Dashboard	`/admin`	Password-gated management interface
Application Modal	`Inline`	Detail view overlay for single application
Navbar	`Global`	Responsive navigation with mobile hamburger

9. DEVELOPMENT ROADMAP

Day 1	Backend Foundation
✓ Node.js + Express.js environment setup with dotenv configuration.	
✓ MongoDB Atlas cluster connection and Mongoose integration.	
✓ Application schema design with validation rules.	
✓ REST API routes: POST /applications, GET /applications/all.	
✓ Resend API integration for automated email notifications.	
✓ CORS configuration for cross-origin frontend communication.	

Day 2	Frontend UI Development
✓ React + Vite project initialization with Tailwind CSS v4.	
✓ Global design system with custom glassmorphic card components.	
✓ Home, Internships, and Apply page development.	
✓ Responsive mobile-first layout implementation.	
✓ React Router DOM v6 navigation setup.	
✓ Axios instance configuration with environment variables.	

Day 3	Integration, Features & Deployment
✓ Frontend-backend integration with full API connectivity.	
✓ Admin Dashboard with search, filter, and pagination.	
✓ Application detail modal with full data display.	
✓ Password-based admin authentication guard.	
✓ Safe array checks and error boundary handling.	
✓ Deployment: Netlify (frontend) + Railway (backend) + environment variables.	

10. CHALLENGES & TESTING

Challenges Faced

CORS Configuration	Resolving cross-origin request blocking between the Netlify frontend and Railway backend required careful configuration of the Express CORS middleware with explicit origin whitelisting.
Environment Variables	Managing VITE_API_URL across development and production environments to ensure correct relative vs. absolute path resolution was critical for deployment stability.
Responsive Glassmorphic UI	Achieving a premium glass-card aesthetic across all device breakpoints required iterative Tailwind CSS customization, especially for backdrop-filter support on Safari.
Safe Array Handling	Preventing dashboard crashes when the applications array was empty or undefined required implementing defensive checks throughout the admin component.

Testing Strategy

Test Type	Tool / Method	Scope
API Testing	Postman v11	All REST endpoints – success & error scenarios
Frontend Validation	Browser / React State	Required fields, email format, form guards
Responsive Testing	Chrome DevTools	iPhone SE, iPad, Desktop (1440px)
Security Testing	Manual	Admin route inaccessible without password
Integration Testing	Manual End-to-End	Full flow: Apply → DB → Email → Dashboard

11. DEPLOYMENT

■ Frontend – Netlify

- React + Vite application deployed to Netlify CDN for global edge delivery.
- Environment variable VITE_API_URL configured in Netlify dashboard.
- Automatic continuous deployment triggered on GitHub repository pushes.
- Custom _redirects file added for React Router SPA routing support.

■ Backend – Railway

- Node.js + Express server hosted on Railway cloud platform.
- Environment variables (MONGO_URI, RESEND_API_KEY, ADMIN_PASSWORD) configured securely.
- Railway auto-detects Node.js and deploys directly from the GitHub repository.
- Railway provides a public HTTPS domain used as the Axios base URL.

Layer	Platform	URL Pattern
Frontend	Netlify	`https://teyzix-core.netlify.app`
Backend	Railway	`https://teyzix-api.up.railway.app`
Database	MongoDB Atlas	`mongodb+srv://cluster0.xxxxx.mongodb.net`
Email	Resend API	`https://api.resend.com/emails`

12. FUTURE IMPROVEMENTS

→ **JWT Authentication**	Replace the current simple password mechanism with JSON Web Token (JWT) based authentication to provide stateless, scalable, and revocable session management.
→ **PDF Resume Upload**	Integrate a file upload system (Cloudinary or AWS S3) to allow applicants to attach PDF resumes to their applications, enriching the admin review process.
→ **Application Status Tracking**	Implement a student-facing status tracker allowing applicants to check their application status (Pending / Under Review / Accepted / Rejected) via email token.
→ **Role-Based Access Control**	Introduce RBAC to support multiple admin roles (Super Admin, Reviewer) with different levels of dashboard access and action permissions.
→ **Analytics Dashboard**	Add a visual analytics panel for administrators showing application trends, domain-wise distribution charts, and submission timeline graphs.

13. CONCLUSION

The **TEYZIX CORE Internship Portal** successfully demonstrates the power of the modern MERN stack in building real-world, production-grade web applications. The project addresses a genuine problem — the inefficiency of manual internship application management — and replaces it with a secure, automated, and visually compelling digital solution.

From backend API design and MongoDB schema modeling to glassmorphic React interfaces and cloud deployment, this project encapsulates the full spectrum of full-stack development. It stands as a testament to applied learning, translating academic knowledge into a tangible, deployable product.

The skills acquired — CORS management, environment variable hygiene, Resend API integration, responsive design, and deployment pipelines — represent a solid foundation for professional software development. The portal is ready for real-world use and is structured for extensibility through the planned future enhancements.

14. REFERENCES

#	Source	URL	Description
[1]	**React Documentation**	`https://react.dev`	Official React framework documentation
[2]	**Node.js Official Site**	`https://nodejs.org`	Node.js runtime reference and guides
[3]	**MongoDB Atlas Guides**	`https://www.mongodb.com/cloud/atlas`	Cloud database setup and best practices
[4]	**Express.js Reference**	`https://expressjs.com`	Express web framework API documentation
[5]	**Axios Documentation**	`https://axios-http.com`	HTTP client library for browser and Node.js
[6]	**Resend API Docs**	`https://resend.com/docs`	Email delivery API documentation
[7]	**Tailwind CSS Docs**	`https://tailwindcss.com/docs`	Utility-first CSS framework reference
[8]	**Mongoose Documentation**	`https://mongoosejs.com/docs`	MongoDB object modeling for Node.js
[9]	**Netlify Documentation**	`https://docs.netlify.com`	Frontend deployment and CDN platform docs
[10]	**Railway Documentation**	`https://docs.railway.app`	Backend cloud deployment platform docs
[11]	**Vite Documentation**	`https://vitejs.dev`	Next generation frontend build tool
[12]	**MDN Web Docs**	`https://developer.mozilla.org`	Web standards and API reference